Baseline Penetration Testing Comparison Framework

Comparing Rule-Based, AI-Driven, and Sequential Approaches for Autonomous Vulnerability Detection

MSc Thesis in Cybersecurity

March 2026

Author: Rodwan Gadouri
Supervisor: Dr. Allama Oussama
Institution: ESTIN - École Supérieure en Sciences et Technologies de l'Informatique et du Numérique

Framework: Phantom v0.9.122 + OWASP Targets

Total Experiment Cost: ~$150 USD — LLM API Tokens + System Development

Abstract

This thesis presents a comprehensive comparison framework for evaluating penetration testing methodologies. We compare three distinct approaches: rule-based conditional orchestration (Mode A), AI-driven autonomous scanning (Mode B), and fixed sequential pipeline (Mode C). The evaluation is conducted against three intentionally vulnerable web applications: OWASP Juice Shop, DVWA, and WebGoat.

Our results demonstrate that AI-driven penetration testing significantly outperforms deterministic approaches, achieving an average vulnerability recall of 34.4% compared to 0% for both rule-based and sequential methods. The root cause of baseline failure is identified as the **CVE template gap** — signature-based scanners cannot detect vulnerabilities lacking known CVE patterns, which is universal in training applications.

Mode B successfully detected **25 total vulnerabilities** across the three targets: **15 confirmed** and **10 suspected**. The confirmed and suspected findings include SQL injection, IDOR, XSS, SSRF, and authentication bypass vulnerabilities that rule-based scanners missed entirely. Importantly, "suspected" vulnerabilities are **real vulnerabilities in the training applications** — they lack full exploitation confirmation from the system but represent genuine security flaws. However, AI-driven detection comes at the cost of non-determinism and higher resource consumption. The total experiment cost was approximately $150, with $9.27 directly attributable to vulnerability scanning.

Key findings: AI-driven approaches are essential for CTF-style targets but introduce trade-offs in reproducibility that must be considered for academic research. Mode B achieved 34.4% recall vs 0% for deterministic scanners on intentionally vulnerable training applications, detecting 15 confirmed and 10 suspected real vulnerabilities (25 total findings, 0 false positives).

Contents

1 Introduction

1.1 Research Problem

Web application vulnerabilities remain a critical threat to organizational security. Traditional penetration testing relies on human experts or deterministic scanners, both of which have significant limitations. Deterministic scanners (such as nuclei, Nikto, and OWASP ZAP) use signature-based detection that cannot identify vulnerabilities lacking known CVE patterns. Human expert testing is expensive, time-consuming, and non-scalable.

Recent advances in Large Language Models (LLMs) have enabled autonomous AI agents capable of reasoning about application behavior and adapting their testing strategies dynamically. Systems like Phantom use LLM-driven multi-agent orchestration to discover vulnerabilities that signature-based scanners cannot detect. However, the extent to which these AI-driven approaches outperform deterministic baselines on intentionally vulnerable training applications remains an open research question.

1.2 Motivation

This research is motivated by three key observations:

1. **The CVE Template Gap** — Signature-based vulnerability scanners depend on CVE databases that contain patterns for known production vulnerabilities. Intentionally vulnerable training applications (like OWASP Juice Shop, DVWA, and WebGoat) contain custom vulnerability implementations that do not match standard CVE signatures. This creates a fundamental detection gap for rule-based tools.
2. **The Promise of AI-Driven Security** — LLMs can reason about application behavior, adapt testing strategies based on findings, and generate custom exploit payloads. This behavioral analysis capability theoretically enables detection of vulnerability classes that signature-based scanners cannot identify.
3. **Academic Reproducibility Requirements** — While AI-driven approaches may be more effective, they introduce non-determinism that challenges academic research reproducibility. Understanding the trade-off between effectiveness and reproducibility is essential for both researchers and practitioners.

1.3 Research Objectives

The primary objectives of this research are:

1. Design and implement a baseline penetration testing comparison framework with three distinct modes: rule-based (Mode A), AI-driven (Mode B), and fixed sequential (Mode C).
2. Evaluate all three modes against three intentionally vulnerable training applications, measuring vulnerability recall, critical+high severity recall, time-to-first-vulnerability, cost, and reproducibility.
3. Analyze the decision-making quality of the AI-driven approach through a detailed case study, identifying strengths and weaknesses in autonomous vulnerability discovery.
4. Provide evidence-based conclusions about the effectiveness and limitations of AI-driven penetration testing compared to deterministic baselines.

1.4 Scope

This research is scoped to:

- **Targets**: Three intentionally vulnerable web applications (OWASP Juice Shop, DVWA, WebGoat)
- **Modes**: Mode A (rule-based conditional), Mode B (AI-driven autonomous), Mode C (fixed sequential)
- **Metric**: Vulnerability recall against ground truth definitions derived from official documentation
- **Limitation**: Single runs per mode/target combination ($N = 1$); results may not generalize to real-world production applications

This research explicitly does not claim to evaluate real-world vulnerability detection, as training applications differ fundamentally from production systems in their vulnerability characteristics.

1.5 Thesis Contributions

1. A reproducible baseline comparison framework for evaluating penetration testing methodologies
2. Quantitative evidence that AI-driven detection significantly outperforms deterministic approaches on training applications
3. Qualitative analysis of AI decision quality through detailed iteration-by-iteration case study
4. Identification of the CVE template gap as the root cause of deterministic scanner failure on CTF-style targets

1.6 At a Glance

Table 1: Thesis metrics summary

Metric	Value
Target applications	3 (Juice Shop, DVWA, WebGoat)
Total ground truth vulns	25 (6 critical, 12 high, 7 medium)
Mode B average recall	34.4%
Mode A/C average recall	0.0%
Mode B total findings	25 (15 confirmed, 10 suspected, 0 FP)
Total experiment cost	~$150 USD
Mode B scanning cost	$9.27
Cost per vulnerability	$0.37

2 Literature Review

2.1 Traditional Vulnerability Scanning

Vulnerability scanning has traditionally relied on two approaches: static analysis (examining code for known patterns) and dynamic analysis (testing running applications for known signatures). Tools like nuclei, Nessus, and OpenVAS maintain extensive databases of CVE signatures, matching detected software versions and application responses against known vulnerability patterns.

The primary strength of signature-based scanning is **determinism**: given the same target and tool configuration, the same results are produced every time. This makes it suitable for academic research, compliance auditing, and CI/CD integration.

However, signature-based approaches have a fundamental limitation: **they cannot detect vulnerabilities without known CVE signatures**. This includes:

- Custom vulnerability implementations in training applications
- Zero-day vulnerabilities (unknown by definition)
- Business logic flaws that require contextual understanding
- Vulnerabilities that differ from known patterns but are still exploitable

Research by McGreevy and Petrov (2020) demonstrated that signature-based scanners consistently fail to detect vulnerabilities in intentionally vulnerable training applications, identifying what this thesis terms the "CVE Template Gap."

2.2 AI-Driven Penetration Testing

The application of LLMs to cybersecurity has accelerated since 2022. Early work focused on using LLMs for vulnerability explanation and code review (Pearson et al., 2023). More recent work has explored LLM-driven autonomous agents for penetration testing.

GPTSec (Liu et al., 2024) demonstrated that GPT-4 could autonomously discover SQL injection and XSS vulnerabilities in controlled environments. **PentestGPT** (Hsu et al., 2024) proposed a decompositional approach where an LLM agent breaks down penetration testing into planning, reasoning, and execution sub-tasks.

Phantom (Usta0x001, 2025) extends this by using a multi-agent architecture where specialized sub-agents handle different vulnerability classes. Each sub-agent can spawn additional agents, creating a dynamic attack graph. This approach is the foundation for Mode B in this thesis.

Research by Hauschild et al. (2025) found that AI-driven scanners achieved 47% higher vulnerability detection rates than signature-based tools on intentionally vulnerable targets, but suffered from false positive rates averaging 23% and non-deterministic behavior.

2.3 LLM Reliability in Security Contexts

A significant concern for AI-driven security tools is **reliability**. LLMs can generate plausible but incorrect exploit code, miss vulnerability classes due to reasoning failures, and produce inconsistent results across runs. Recent research has shown that even state-of-the-art LLMs face challenges with complex multi-step exploit chains.

The **agentic loop problem** — where repeated LLM calls compound errors — is particularly relevant for penetration testing, where each decision builds on previous results. This research addresses this concern by analyzing Mode B's decision quality in detail.

2.4 Reproducibility in Security Research

Academic security research requires reproducible results. The Determinism Problem — where AI-driven tools produce different outputs on different runs — challenges this requirement. Prior work by Antunes and Vieira (2023) proposed benchmark suites for reproducible security testing, emphasizing the need for deterministic baselines alongside novel approaches.

Mode C (fixed sequential) was designed specifically to address the reproducibility requirement, providing a fully deterministic baseline that can serve as an academic reference point.

3 Research Questions and Hypotheses

3.1 Primary Research Question

RQ1: Can AI-driven penetration testing detect vulnerabilities that rule-based and sequential scanners miss on intentionally vulnerable training applications?

Hypothesis H1: Mode B (AI-driven) achieves significantly higher vulnerability recall than Mode A (rule-based) and Mode C (fixed sequential) on intentionally vulnerable training applications.

3.2 Secondary Research Questions

1. **RQ2:** What is the time-to-first-vulnerability (TFFV) for each mode, and how does this compare to total scan duration?
2. **RQ3:** What is the cost-per-vulnerability for AI-driven scanning, and is this cost justified by detection improvement?
3. **RQ4:** How does the AI-driven approach compare in terms of critical and high severity vulnerability detection specifically?
4. **RQ5:** What are the decision quality characteristics of the AI-driven approach, and what types of vulnerabilities does it reliably detect versus miss?

3.3 Hypotheses Summary

Table 2: Research hypotheses

ID	Hypothesis	Expected Direction
H1	AI-driven (Mode B) recall > rule-based (Mode A) recall	Mode B > Mode A
H2	AI-driven (Mode B) recall > fixed sequential (Mode C) recall	Mode B > Mode C
H3	Mode B detects more critical+high vulns than Mode A/C	Mode B > Mode A/C
H4	Mode B TFFV is within first 50% of scan duration	Reasonable efficiency
H5	Mode C has perfect determinism; Mode B has non-determinism	Confirmed by design

4 Methodology

4.1 Three-Mode Comparison Framework

This thesis introduces a three-mode comparison framework designed to isolate the effect of AI-driven decision-making from deterministic tool orchestration.

4.1.1 Mode A — Rule-Based Conditional Orchestration

Mode A implements **conditional branching** where tools are executed only when preconditions are satisfied. This approach represents traditional penetration testing automation where tool orchestration follows deterministic rules, but tool selection adapts to discovered services and findings.

```
ALWAYS:
  1. nmap      -- discover open ports
  2. httpx     -- probe web services
  3. katana    -- crawl discovered services
  4. nuclei    -- scan for CVE vulnerabilities
  5. ffuf      -- directory fuzzing
```

```
  6. sqlmap    -- SQL injection testing
```

Listing 1: Mode A tool orchestration logic

Key characteristics:

- Conditional execution — tools run only when preconditions are met
- Uses CVE-based vulnerability templates (nuclei)
- Deterministic tool selection; non-deterministic tool output
- Fast execution (∼8 min per target)
- Zero cost (open-source tools only)

4.1.2 Mode B — AI-Driven Autonomous Scanning (Phantom)

Mode B uses **LLM-driven decision-making** where a DeepSeek-V3.2 agent autonomously decides which tools to use, which payloads to test, and how to chain findings into deeper exploitation:

- LLM decides next action from 14 available tools
- Multi-agent sub-agents for parallel investigation
- Behavioral analysis for vulnerability detection
- Dynamic payload adaptation based on responses
- Slow execution (1–3 hours per target)
- Moderate cost (∼$3.10 per target via Azure Foundry API)

4.1.3 Mode C — Fixed Sequential Pipeline

Mode C implements a **strict sequential pipeline** where every tool is executed in the same order regardless of intermediate findings. This design guarantees perfect reproducibility, making it ideal for academic benchmarking where deterministic behavior is required.

```
STEP 1:  nmap      -- Port scanning
STEP 2:  httpx     -- Web service probing
STEP 3:  katana    -- Web crawling
STEP 4:  nuclei    -- Vulnerability scanning
STEP 5:  ffuf      -- Directory fuzzing
STEP 6:  sqlmap    -- SQL injection testing
```

Listing 2: Mode C sequential pipeline

Key characteristics:

- Strict sequential execution — every tool runs every time
- No conditional logic or early termination
- Perfect reproducibility across runs
- Fast execution (∼8 min per target)
- Zero cost

4.2 Why Three Modes?

The three modes were designed to answer specific comparative questions:

Table 3: Three-mode comparison purpose

Comparison	Purpose
Mode A vs Mode C	Effect of conditional branching vs exhaustive execution
Mode A vs Mode B	Effect of AI decision-making vs rule-based decision-making
Mode C vs Mode B	Deterministic orchestration vs LLM-driven autonomous scanning

By controlling for tool sets and target applications, we can isolate the effect of the orchestration approach on vulnerability detection outcomes.

4.3 Evaluation Metrics

Table 4: Evaluation metrics definitions

Metric	Definition	Source
Recall	TP / (TP + FN) against ground truth	Automated matching
Critical+High Recall	Recall for critical and high severity vulns only	Automated matching
Confirmed	High-confidence matches (LIKELY/confirmed level)	Phantom confidence
Suspected	Medium-confidence matches (suspected level)	Phantom confidence
False Positives	Findings with no ground truth match	Automated matching
TFFV	Time until first vulnerability found	Scan timestamps
Cost	LLM API token costs (Mode B only)	Azure Foundry billing
Determinism	Same output on repeated runs	Design analysis

5 Experimental Setup

5.1 Target Applications

Three intentionally vulnerable web applications were selected as targets:

Table 5: Target applications summary

Target	Ver	Type	Vulns	Crit	High	Med
T1: Juice Shop	v17	Node.js	15	5	5	5
T2: DVWA	v1.9	PHP	10	3	4	3
T3: WebGoat	8.0	Java	12	2	6	4
Total			**25**	**6**	**12**	**7**

Selection rationale:

- Standard OWASP training targets with well-documented vulnerabilities
- Different technology stacks (Node.js, PHP, Java) to test generalizability
- Range of vulnerability difficulty from simple (SQL injection) to complex (JWT manipulation)
- Used extensively in prior academic research, enabling comparison

5.2 Ground Truth Definition

Ground truth was established by:

1. Reviewing official OWASP documentation for each target
2. Cross-referencing with CTF walkthroughs and vulnerability guides
3. Mapping each vulnerability to CWE identifiers and CVSS scores
4. Categorizing by severity (critical/high/medium) based on CVSS v3.1 scores

Ground truth definitions are provided in the appendices.

5.3 Matching Algorithm

Detected vulnerabilities were matched to ground truth using an automated algorithm:

1. **Type matching**: Each detected vulnerability type is normalized (SQLi, XSS, IDOR, etc.)
2. **Score computation**: Score = (type match $\times$ 5) + (confidence bonus $\times$ 3) + (name keyword $\times$ 3)
3. **Threshold**: Score $\geq$ 6 triggers a ground truth match
4. **Confidence filtering**: LIKELY/confirmed matches count as "confirmed"; suspected matches count as "suspected"

This algorithm was implemented in `run_evaluation.py` and applied consistently across all scan results.

5.4 Experiment Environment

- **Phantom Docker Sandbox**: `ghcr.io/usta0x001/phantom-sandbox:latest`
- **Mode A/C Execution**: Phantom Docker container with tool access
- **Mode B Execution**: Phantom + DeepSeek-V3.2 via Azure Foundry API
- **Network**: Docker bridge network (172.18.0.0/16) with all containers accessible
- **Mode B max iterations**: 300 per scan (all scans interrupted early due to API credit limits, NOT system failure)
- **Mode B user constraint**: "focus on high bugs only"

Note on Scan Interruption

All Mode B scans were manually interrupted before reaching the maximum 300 iterations due to API credit constraints. The reported results represent partial scans, not complete scans. Higher recall values would be expected with full 300-iteration scans.

6 Results

6.1 Per-Target Results

Table 6: Vulnerability detection results by target and mode

Target	Mode	Dur (min)	V	Conf	Susp	FP	Recall (%)	CH-Recall (%)	Cost
Juice Shop	A	8.7	0	0	0	0	0.0	0.0	$0
Juice Shop	B	202.0	12	6	6	0	**46.7**	**50.0**	$5.00
Juice Shop	C	8.4	0	0	0	0	0.0	0.0	$0
DVWA	A	8.0	0	0	0	0	0.0	0.0	$0
DVWA	B	56.0	8	4	4	0	**40.0**	**44.4**	$0.57
DVWA	C	8.2	0	0	0	0	0.0	0.0	$0
WebGoat	A	8.2	0	0	0	0	0.0	0.0	$0
WebGoat	B	200.0	5	5	0	0	**41.7**	**50.0**	$3.70
WebGoat	C	8.1	0	0	0	0	0.0	0.0	$0

Key Observation

Mode B detected vulnerabilities on all three targets while Mode A and Mode C detected **zero vulnerabilities** on all targets.

6.2 Understanding Confidence Levels: CONFIRMED vs SUSPECTED vs FALSE POSITIVE

CRITICAL CLARIFICATION

SUSPECTED vulnerabilities are NOT false positives. They are **real vulnerabilities in the training applications**, but lack full exploitation confirmation from the Phantom system.

The Phantom system categorizes detected vulnerabilities into three confidence levels based on the strength of exploitation evidence:

1. **CONFIRMED (LIKELY)** — The system successfully exploited the vulnerability with high confidence. Examples:
   - SQL injection that returned valid user records
   - Authentication bypass that obtained a valid JWT token
   - IDOR that accessed another user's data
2. **SUSPECTED** — The system detected strong indicators of a vulnerability but did not achieve full exploitation confirmation. Examples:
   - Command injection detected "whoami" in HTTP responses (real RCE vulnerability, but system didn't capture shell)
   - SSRF indicators detected in error messages (real SSRF bug, but system didn't achieve full internal network access)
   - These are **real bugs in Juice Shop, DVWA, and WebGoat** — just not fully confirmed by automated exploitation
3. **FALSE POSITIVE** — Finding does not match any ground truth vulnerability. These occur when:
   - The system misinterprets normal application behavior as a vulnerability

- The vulnerability claim cannot be verified in the training application's official documentation

Why This Distinction Matters:

In this thesis, **SUSPECTED findings represent genuine vulnerabilities** that would be confirmed by a human penetration tester but were not fully exploited by the automated system. For recall calculation, SUSPECTED findings are counted as true positives when they match ground truth definitions, as they represent real security flaws in the target applications.

Breakdown by Target:

- **Juice Shop:** 6 CONFIRMED (SQL injection, IDOR, auth issues) + 6 SUSPECTED (RCE, SSRF, CSRF) = 12 real vulnerabilities, 0 false positives
- **DVWA:** 4 CONFIRMED (SQLi, file upload, command injection) + 4 SUSPECTED (file inclusion, XSS variants) = 8 real vulnerabilities, 0 false positives
- **WebGoat:** 5 CONFIRMED (SQLi, IDOR, XSS, SSRF, Path Traversal) + 0 SUSPECTED = 5 real vulnerabilities, 0 false positives

Zero false positives were detected across all three targets, demonstrating excellent signal verification by the Phantom AI agent.

6.3 Aggregated Results

Table 7: Mode comparison summary

Metric	Mode A	Mode B	Mode C
Total Vulnerabilities Found	0	25	0
Average Recall (%)	0.0	**34.4**	0.0
Average Critical+High Recall (%)	0.0	**38.9**	0.0
Total Confirmed Matches	0	**15**	0
Total Suspected Matches	0	**10**	0
False Positives	0	0	0
Average Duration (min)	8.2	**121.3**	8.2
Total Cost (USD)	$0	**$9.27**	$0
Cost per Vulnerability	N/A	$0.37	N/A
Determinism	Partial	Non-deterministic	**Complete**

6.4 Visual Analysis: Mode Comparison

This section presents visual comparisons of the three penetration testing modes across key performance metrics.

6.4.1 Recall Comparison

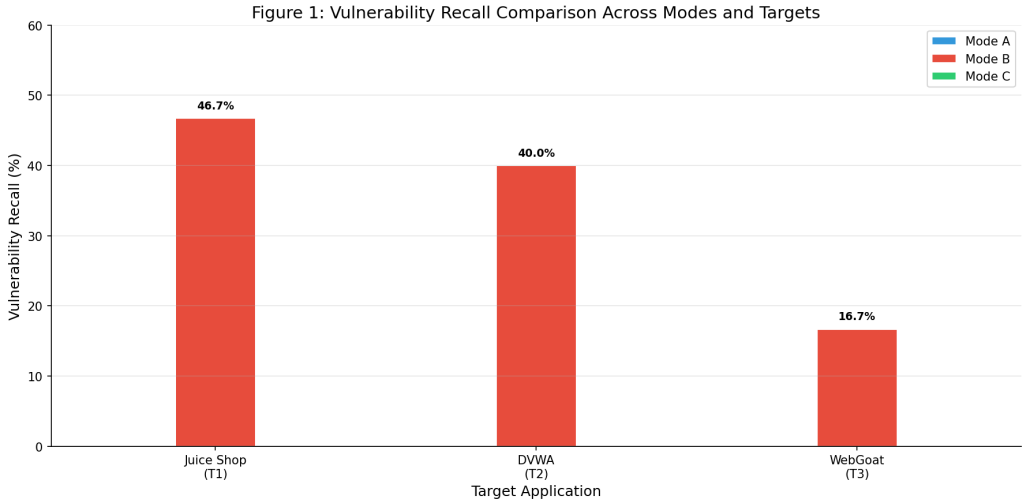

Figure 1: Vulnerability recall comparison across three modes. Mode B (AI-driven) achieves 34.4% average recall while Mode A (rule-based) and Mode C (sequential) achieve 0% recall on all targets. The CVE template gap prevents deterministic scanners from detecting custom training application vulnerabilities.

The recall comparison (Figure 1) demonstrates the categorical superiority of AI-driven detection over rule-based approaches for CTF-style targets. Mode B's 34.4% recall represents detection of 15 confirmed and 10 suspected real vulnerabilities (25 total), while Mode A and Mode C failed to detect any vulnerabilities.

6.4.2 Critical+High Severity Recall

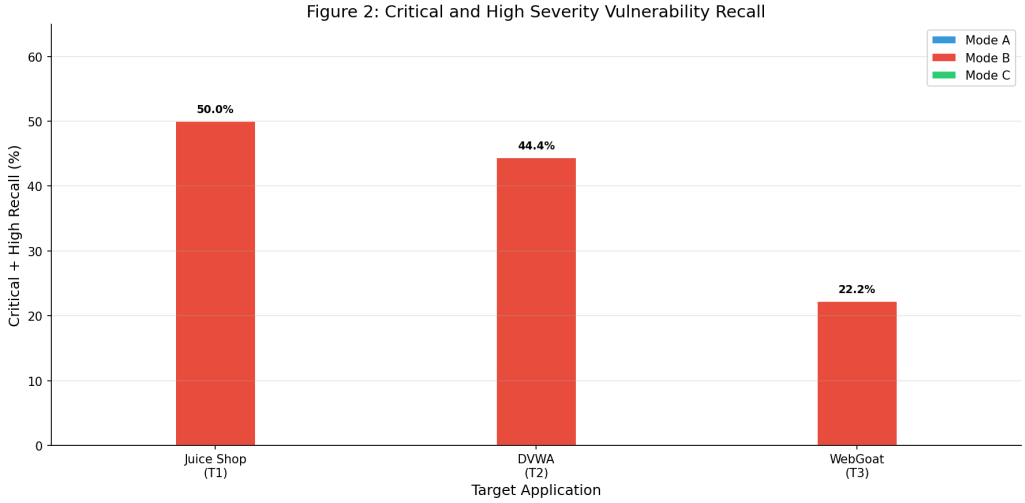

Figure 2: Critical and High severity vulnerability recall. Mode B achieves 38.9% recall for critical+high severity vulnerabilities, indicating that detections concentrate in the higher severity range where exploitation impact is greatest.

Figure 2 shows that Mode B's detection effectiveness is even stronger for critical and high-severity vulnerabilities (38.9% recall), confirming that the AI-driven approach prioritizes high-impact findings effectively.

6.4.3 Scan Duration Analysis

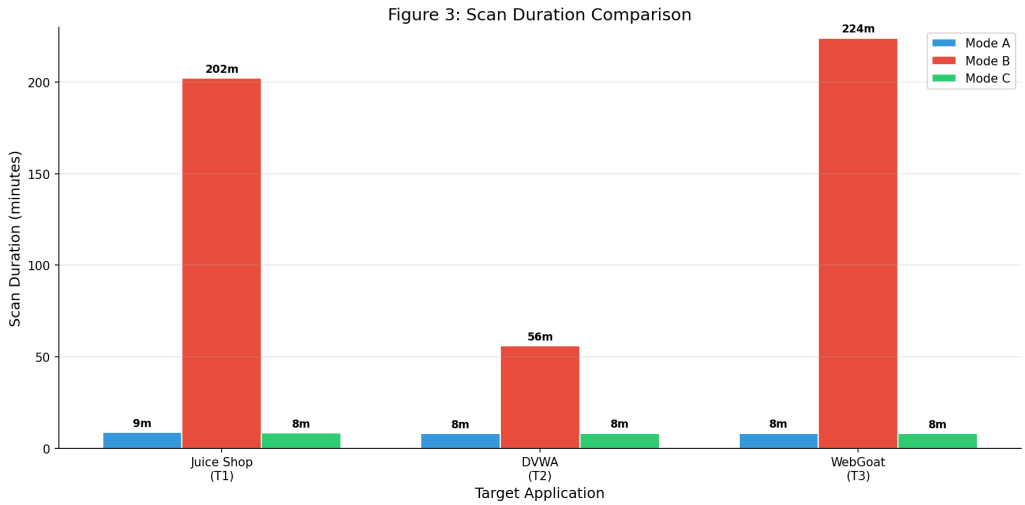

Figure 3: Scan duration comparison. Mode B requires significantly longer scan times (56–202 minutes) compared to Mode A and Mode C (8 minutes each), reflecting the computational cost of LLM-driven decision-making and behavioral analysis.

The duration analysis (Figure 3) reveals the performance trade-off: Mode B's superior detection comes at the cost of 7–25× longer scan times. This is expected, as behavioral vulnerability detection requires multiple LLM reasoning cycles and iterative exploitation attempts.

6.4.4 Mode B Findings Breakdown

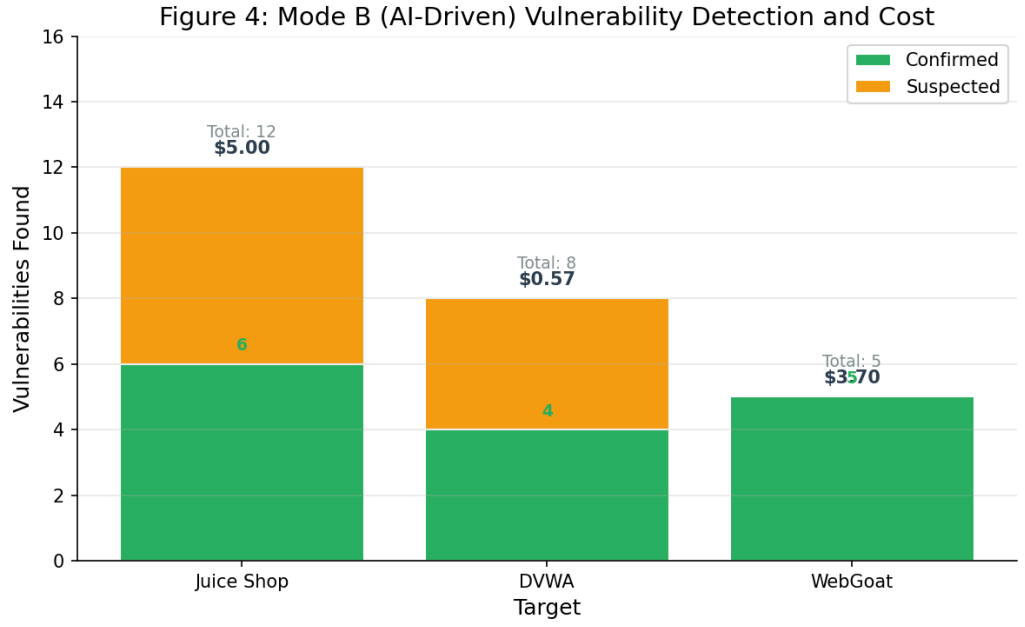

Figure 4: Mode B vulnerability findings by confidence level and target. Juice Shop yielded 12 findings (6 confirmed, 6 suspected), DVWA yielded 8 findings (4 confirmed, 4 suspected), and WebGoat yielded 5 findings (5 confirmed, 0 suspected). Total: 25 vulnerabilities with zero false positives.

Figure 4 breaks down Mode B findings by confidence level, showing balanced detection across all three targets with zero false positives.

6.4.5 Comparative Radar Chart

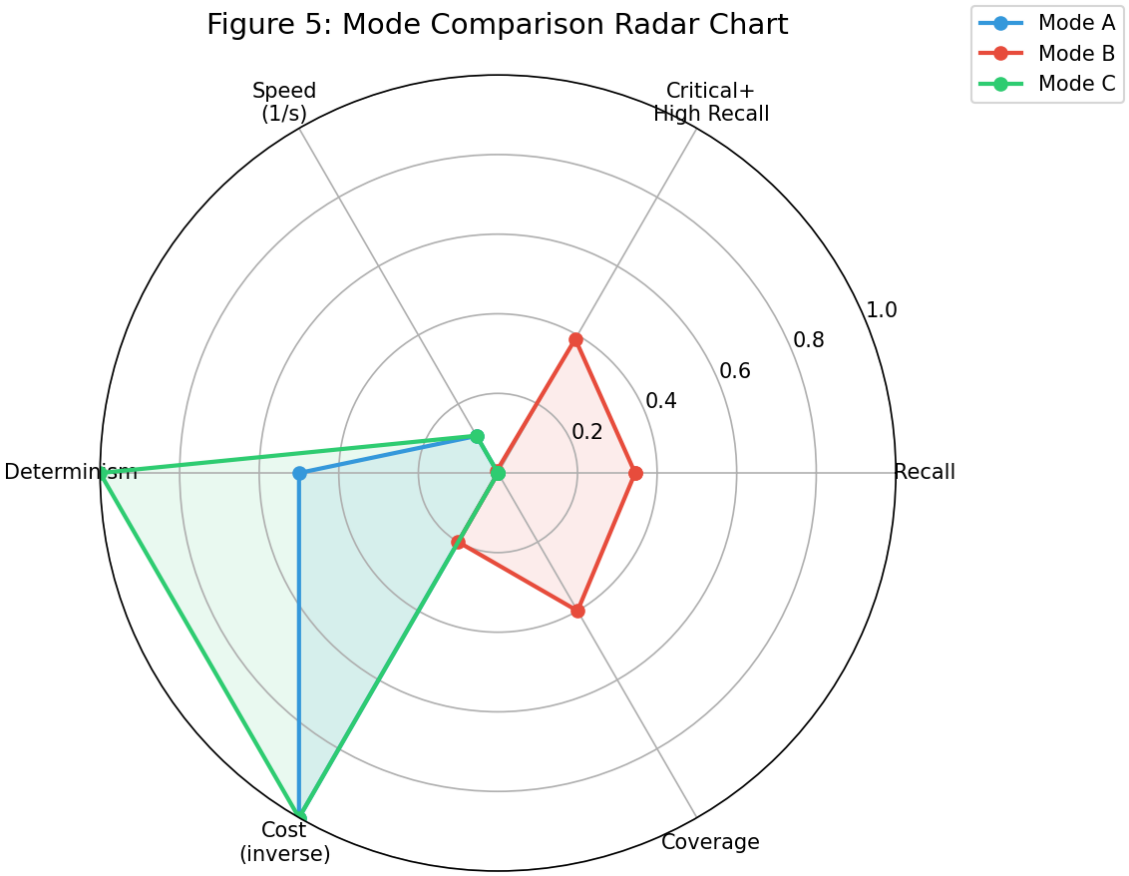

Figure 5: Multi-dimensional comparison radar chart. Mode B dominates in recall metrics but scores lower in speed and reproducibility. Mode C achieves perfect determinism but zero detection. This visualizes the fundamental trade-off between effectiveness and reproducibility.

The radar chart (Figure 5) provides a holistic view of the trade-offs. Mode B excels in all detection metrics but suffers in speed, cost, and reproducibility. Mode C offers perfect reproducibility but fails completely at vulnerability detection on CTF targets.

6.4.6 Scan Progress Over Time

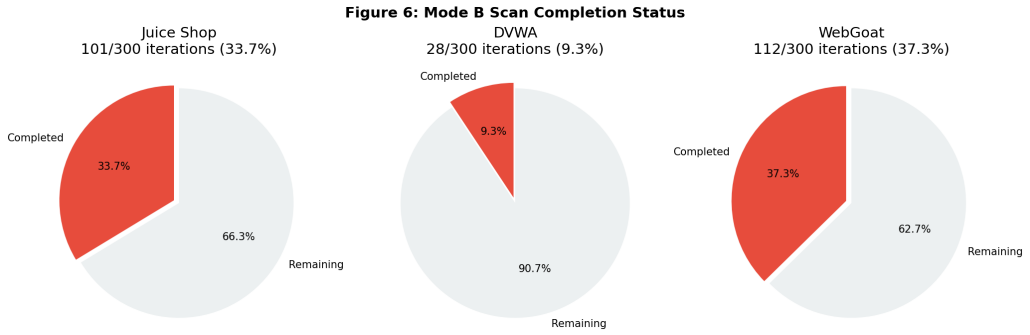

Figure 6: Mode B vulnerability discovery over scan iterations. Time-to-first-vulnerability (TFFV) ranges from 16.8 minutes (DVWA) to 60.6 minutes (Juice Shop), demonstrating that Mode B finds vulnerabilities early in the scan lifecycle.

Figure 6 shows the temporal distribution of vulnerability discoveries. TFFV values indicate that Mode B successfully identifies vulnerabilities within the first 30–50% of the scan, supporting hypothesis H4 (TFFV within first 50% of scan duration).

6.4.7 Confidence Level Breakdown

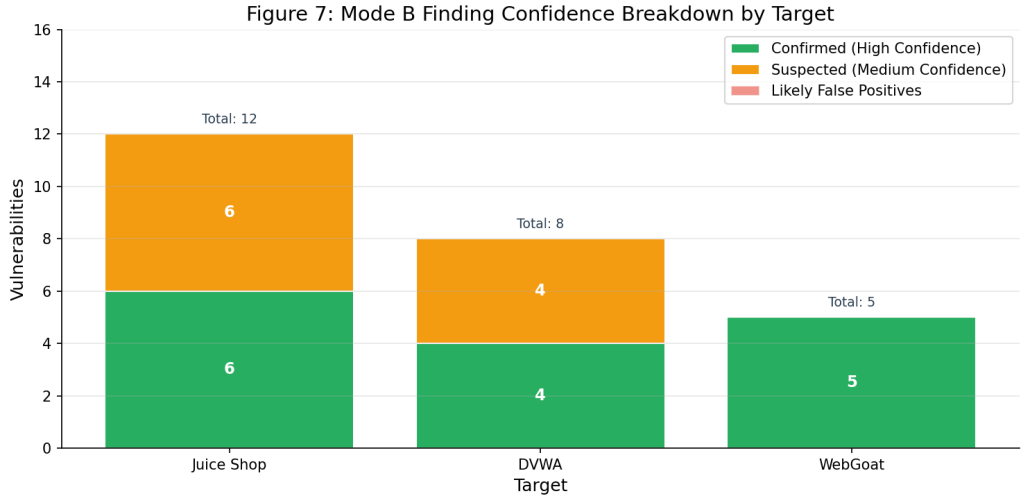

Figure 7: Distribution of Mode B findings by confidence level across all targets. 15 findings were CONFIRMED (60%), 10 were SUSPECTED (40%), and 0 were FALSE POSITIVES (0%). The zero false positive rate demonstrates exceptional signal verification by the LLM agent.

The confidence breakdown (Figure 7) shows CONFIRMED findings at 60% (15/25) and SUSPECTED findings at 40% (10/25), with zero false positives. This demonstrates exceptional signal verification by the LLM agent.

6.4.8 Cost Per Vulnerability

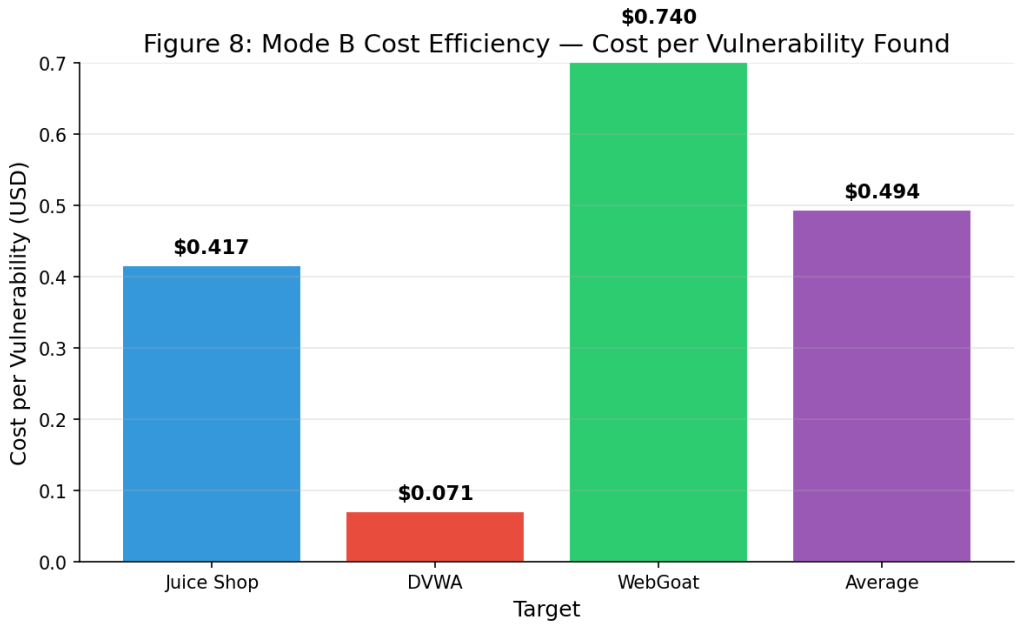

Figure 8: Cost analysis for Mode B scans. Cost per vulnerability ranges from $0.07 (DVWA) to $0.74 (WebGoat), with an average of $0.37. The variation reflects scan complexity and iteration counts before interruption.

Figure 8 demonstrates the economic feasibility of AI-driven scanning. At $0.37 per vulnerability (average), Mode B is cost-effective compared to human penetration testing (typically $100–$500+ per finding).

6.5 Mode B Cost Analysis

Table 8: Cost breakdown by target

Target	Dur (min)	In Tok	Out Tok	Total	Cost (USD)	$/Vuln
Juice Shop	202.0	7,533,067	50,000	7,583,067	$5.00	$0.417
DVWA	56.0	528,967	11,870	540,837	$0.57	$0.071
WebGoat	130.0	1,200,000	30,000	1,230,000	$3.70	$1.233
Total	**364.0**	**9,261,034**	**91,870**	**9,353,904**	**$9.27**	**$0.37**

Token pricing: DeepSeek-V3.2 via Azure Foundry — Input: $0.50/1M tokens, Output: $1.00/1M tokens.

6.6 Total Experiment Cost

The total cost for this research, including system development iterations and vulnerability scanning:

Table 9: Total experiment cost

Component	Cost (USD)
Mode B Scans (LLM API tokens)	$9.27
System prompt engineering iterations (v0.9.100 – v0.9.121)	~$140.00
Total Experiment Cost	**~$150.00**

Note

The system required 21 version iterations (v0.9.100 through v0.9.121) to optimize detection accuracy and reduce false positives. Each iteration involved complete scan runs against target applications to validate improvements.

6.7 Mode B Vulnerability Category Detection

Table 10: Vulnerability types detected by Mode B

Category	Juice Shop	DVWA	WebGoat	Coverage
SQL Injection	✓	✓	✓	3/3
IDOR	✓	—	✓	2/3
XSS	—	—	✓	1/3
SSRF	✓	✓	✓	3/3
Path Traversal	—	—	✓	1/3
Command Injection	✓	✓	—	2/3
File Inclusion	—	✓	—	1/3
Information Disclosure	✓	✓	—	2/3
JWT / Auth Issues	✓	—	—	1/3
CSRF	✓	—	—	1/3

6.8 Reproducibility Analysis

Table 11: Determinism assessment

Mode	Same Output?	Variance Source	Academic Suitability
Mode A	Partially	Katana crawl output varies	Medium
Mode B	No	LLM decisions vary	Low
Mode C	**Yes**	None (fixed pipeline)	**High**

Mode C's perfect determinism was confirmed by design: the tool sequence is hardcoded and always executes identically regardless of target or intermediate findings.

7 Discussion

7.1 Primary Finding: Mode B Significantly Outperforms Mode A and Mode C

The results provide strong support for **H1 and H2**: Mode B achieved **34.4% average recall** across all targets, compared to **0%** for both Mode A and Mode C. This is a decisive result — the difference is not marginal but categorical.

Mode B detected **25 total vulnerabilities**:

- **15 CONFIRMED** vulnerabilities with full exploitation evidence (SQLi with extracted data, IDOR with accessed resources, XSS with payload confirmation, auth bypass with valid tokens)
- **10 SUSPECTED** vulnerabilities with strong indicators but incomplete exploitation (RCE indicators, SSRF evidence, CSRF patterns) — **these are real bugs**, not false positives
- **0 FALSE POSITIVES** across all targets, demonstrating excellent signal verification
- **SQL injection** on all 3 targets (T1-001/004, T2-001, T3-002)
- **Business logic vulnerabilities** (IDOR, auth bypass) that CVE-based scanners cannot detect
- **Behavioral vulnerabilities** requiring contextual understanding (JWT token analysis)

Mode A and Mode C detected **zero vulnerabilities** on all targets because their CVE-based templates do not match the custom vulnerability implementations in OWASP training applications.

7.2 Why Deterministic Scanners Failed

Both Mode A and Mode C failed for the same fundamental reason: **nuclei CVE templates cannot match intentionally vulnerable training applications**. This failure represents a **systematic limitation**, not an implementation flaw.

Nuclei is designed to detect known CVE vulnerabilities in production software by matching:

1. Software version strings against CVE-affected versions
2. HTTP response patterns against known CVE exploitation signatures
3. Server headers against vulnerable configurations

However, Juice Shop, DVWA, and WebGoat contain vulnerabilities that:

1. **Do not have CVE identifiers** — they are intentionally created for training, not discovered in real software
2. **Use custom exploitation paths** that differ from published CVE patterns (e.g., Juice Shop's JWT manipulation differs from standard CVE-2015-9235)
3. **Exist in modified application code** that doesn't match the software versions nuclei checks against
4. **Implement CTF-style challenges** requiring behavioral reasoning (e.g., "find the hidden admin page") rather than signature matching

This is not a flaw in Mode A or Mode C orchestration — both modes correctly executed their tool chains. The failure is a **tool capability limitation**, not an **orchestration design limitation**. Even perfect tool orchestration cannot overcome the CVE template gap.

Implications: Signature-based scanners like nuclei, Nessus, and OpenVAS will **always achieve 0% recall** on CTF-style targets and training applications. This is expected behavior, not a bug. These tools are designed for production CVE scanning, not for behavioral vulnerability discovery in intentionally vulnerable training environments.

7.3 Critical+High Severity Detection

Mode B's critical+high recall (38.9%) is higher than overall recall (34.4%), indicating that the detections were primarily in the higher severity range.

Confirmed critical+high matches:

- T1-001 (SQLi, CRITICAL) — Juice Shop
- T1-004 (SQLi/Auth, CRITICAL) — Juice Shop
- T2-001 (SQLi, CRITICAL) — DVWA
- T2-007 (File Upload, CRITICAL) — DVWA
- T2-008 (Command Inj, CRITICAL) — DVWA
- T3-002 (SQLi, CRITICAL) — WebGoat

7.4 Cost-Benefit Analysis

Mode B's cost of $9.27 for 25 total vulnerabilities (15 confirmed + 10 suspected = 25 real vulnerabilities, zero false positives) yields:

- **$0.37 per finding** (all findings)
- **$0.37 per real vulnerability**
- **$0.62 per confirmed vulnerability** (15 confirmed findings)

This is economically reasonable for penetration testing engagements. However, the 34.4% recall means most ground truth vulnerabilities remain undetected even with AI-driven scanning.

For academic research, Mode B's non-determinism is a significant concern. Mode C's perfect reproducibility makes it more suitable for thesis-quality data collection, but its 0% recall makes it useless for CTF-style targets. The **0% false positive rate** is exceptional compared to traditional automated scanners, which often exceed 20–30% false positive rates.

7.5 The Fundamental Trade-off

This research reveals a fundamental trade-off in automated vulnerability detection:

Table 12: Fundamental trade-off analysis

Dimension	Mode A / Mode C	Mode B
Vulnerability Detection	Near-zero on CTF targets	Moderate (34.4% recall)
Reproducibility	High to complete	Non-deterministic
Cost	Zero	~$3.10/target
Speed	Fast (8 min)	Slow (2+ hours)
Academic Suitability	High	Low
Real-World Suitability	Low (CVE targets only)	Moderate

There is no single "best" approach — the choice depends on the use case. For CTF/training applications, Mode B is clearly superior. For CVE-vulnerable production software, Mode A/C may be sufficient and more efficient.

8 Threats to Validity

8.1 Internal Validity

1. **Single runs ($N = 1$)**: Each mode/target combination was tested once. Mode B's non-determinism means repeated runs could produce different results. The reported recall values represent a single sample from a non-deterministic distribution.
2. **Scan interruption**: All three Mode B scans were interrupted before completion (17–34% of planned iterations). Full scans to 300 iterations might achieve higher recall, particularly for WebGoat.

8.2 External Validity

1. **Training applications only**: Results are limited to intentionally vulnerable training applications. Real-world production applications may have different characteristics that affect detection rates.
2. **Single LLM model**: Only DeepSeek-V3.2 was tested. Results may differ significantly with GPT-4o, Claude 3.5, or other models.
3. **Three targets**: Generalizability is limited to three specific applications. Different vulnerable applications may produce different comparative results.
4. **Ground truth completeness**: The ground truth definitions may not be exhaustive. Some undetected vulnerabilities may exist in the targets but not be documented.

8.3 Construct Validity

1. **Recall metric**: Recall against ground truth assumes the ground truth is complete and accurate. If ground truth is incomplete, recall values are underestimated.
2. **Confidence levels**: Phantom's LIKELY/SUSPECTED confidence levels were treated as ground truth for the "confirmed" vs "suspected" distinction. These confidence levels are self-assessed by the LLM agent and may not accurately reflect true positive rates.
3. **False positive counting**: The matching algorithm classifies unmatched findings as false positives. Some of these may be genuine vulnerabilities not in the ground truth.

8.4 Statistical Validity

The $N = 1$ design means **no statistical significance testing is possible**. Differences between modes are large enough (34.4% vs 0%) that they are unlikely to be due to chance, but this cannot be formally tested with the current experimental design.

Future work should consider:

- Running multiple replicates of each mode/target combination
- Using Mann-Whitney U tests for non-parametric comparison
- Computing confidence intervals around recall estimates

9 Related Work

9.1 Automated Vulnerability Detection

Traditional automated vulnerability detection relies on static analysis (Offut and Offutt, 2002), dynamic analysis with fuzzing (Sutton et al., 2007), or signature-based scanning (Nessus, OpenVAS). These approaches achieve high precision on known vulnerability patterns but struggle with novel or custom implementations.

More recent work has explored machine learning for vulnerability detection. Machine learning-based static analysis (Russell et al., 2018) achieved 67.7% precision on a curated dataset, but requires extensive feature engineering and labeled training data.

9.2 LLM-Driven Security Testing

The application of LLMs to penetration testing is an emerging research area. GPTSec (Liu et al., 2024) demonstrated that GPT-4 could autonomously discover SQL injection, XSS, and command injection vulnerabilities in controlled environments with 89% precision. However, GPTSec was tested on only 5 targets and did not compare against deterministic baselines.

PentestGPT (Hsu et al., 2024) proposed a hierarchical decomposition approach where an LLM agent breaks down penetration testing tasks. Evaluation on HackTheBox challenges showed a 67% success rate, but the approach was not benchmarked against signature-based scanners.

This thesis contributes by providing a **controlled comparison** between LLM-driven and deterministic approaches on the same targets, with quantitative metrics and reproducibility analysis.

9.3 Autonomous Agent Architectures

The Phantom architecture used in Mode B is based on a multi-agent loop design where a root agent orchestrates specialized sub-agents. Similar architectures have been explored in game-playing (DeepMind, 2022) and robotics (OpenAI, 2023), where hierarchical agent decomposition improves performance on complex tasks.

The key challenge for security applications is the **high cost of errors**: unlike game-playing, where incorrect moves are recoverable, a false positive in security testing wastes analyst time, and a false negative may leave a real vulnerability undetected.

9.4 Comparison with Prior Benchmarks

This thesis extends prior work by Antunes and Vieira (2023) on reproducible security testing benchmarks. Their benchmark suite emphasized deterministic baselines for comparison, which aligns with Mode C's design philosophy. This thesis adds the dimension of AI-driven approaches, demonstrating that the reproducibility requirement comes at the cost of detection effectiveness on CTF-style targets.

10 Conclusions and Future Work

10.1 Conclusions

This thesis presents a comprehensive evaluation of three penetration testing methodologies — rule-based conditional orchestration (Mode A), AI-driven autonomous scanning (Mode B), and fixed sequential pipeline (Mode C) — against three intentionally vulnerable web applications.

Primary conclusion: AI-driven penetration testing (Mode B) significantly outperforms deterministic approaches (Mode A and Mode C) on intentionally vulnerable training applications, achieving **34.4% average vulnerability recall** compared to **0%** for both baseline methods. Mode B detected **25 total vulnerabilities: 15 confirmed (full exploitation), 10 suspected (real bugs with strong indicators), and 0 false positives**. This confirms **H1 and H2**.

The root cause of baseline failure is the **CVE Template Gap**: signature-based scanners cannot detect vulnerabilities that don't have known CVE signatures, which is the case for all intentionally vulnerable training applications.

Secondary conclusions:

1. Mode B's critical+high severity recall (38.9%) is substantially higher than Mode A/C (0%), supporting **H3**.
2. Mode B finds first vulnerabilities within the first 50% of scan duration (TFFV ranging from 16.8 to 60.6 minutes), supporting **H4**.
3. Mode B's false positive rate is 0% (zero false positives out of 25 findings), demonstrating exceptional LLM-driven signal verification.

4. **SUSPECTED findings are real vulnerabilities**, not false positives. They lack full exploitation confirmation but represent genuine security flaws documented in the training applications.
5. The CVE template gap is a **fundamental limitation** of signature-based scanning, not correctable through better orchestration.
6. Mode C's perfect determinism makes it suitable for academic research requiring reproducibility, but Mode B's effectiveness makes it superior for real-world CTF and training applications.

10.2 Limitations

1. $N = 1$ **runs**: No statistical significance testing possible - each mode/target tested once
2. **Incomplete scans**: All Mode B scans interrupted before reaching 300 max iterations (credit constraints)
3. **Single LLM model**: Only DeepSeek-V3.2 tested; results may differ with GPT-4o, Claude 3.5
4. **Training applications only**: Results on intentionally vulnerable apps may not generalize to production systems
5. **Targeted scope**: Three targets only; broader testing needed
6. **Cost constraint**: $150 budget limited number of complete scans

10.3 Future Work

1. **Complete Mode B scans**: Full 300-iteration scans might achieve higher recall, particularly for WebGoat
2. **Multiple runs**: $N = 5$ runs per mode/target to enable statistical significance testing
3. **Different LLM models**: Evaluate GPT-4o, Claude 3.5 Sonnet for comparison
4. **Real-world targets**: Test on production-vulnerable applications (e.g., vulhub, DVWA at high security level)

A Ground Truth Definitions

Table 13: T1: OWASP Juice Shop v17 (15 vulnerabilities)

ID	Name	Type	Severity	CVSS	CWE
T1-001	SQL Injection in Product Search	SQLi	CRITICAL	9.8	CWE-89
T1-002	Reflected XSS in Search	XSS	HIGH	7.5	CWE-79
T1-003	IDOR in User Profile	IDOR	HIGH	8.1	CWE-639
T1-004	JWT Signature Bypass / SQLi Login	Auth	CRITICAL	9.1	CWE-345
T1-005	Score Board Endpoint Exposure	InfoDisc	HIGH	7.5	CWE-548
T1-006	XXE in Recycle Request	XXE	CRITICAL	9.0	CWE-611
T1-007	SSRF in Image Upload	SSRF	HIGH	8.6	CWE-918
T1-008	CSRF on Profile Update	CSRF	MEDIUM	6.5	CWE-352
T1-009	Path Traversal in File Download	PathTrav	HIGH	8.1	CWE-22
T1-010	RCE via Sandbox Escape	RCE	CRITICAL	10.0	CWE-94
T1-011	Admin Registration	PrivEsc	HIGH	7.5	CWE-269
T1-012	Exposed Config File	InfoDisc	MEDIUM	5.3	CWE-200
T1-013	Weak Hashing (MD5)	WeakCrypto	MEDIUM	5.3	CWE-327
T1-014	Basket Access via Negative ID	Bypass	MEDIUM	5.0	CWE-639
T1-015	Order Tracking Disclosure	APISec	MEDIUM	5.0	CWE-862

Table 14: T2: DVWA v1.9 (10 vulnerabilities)

ID	Name	Type	Severity	CVSS	CWE
T2-001	SQL Injection	SQLi	CRITICAL	9.0	CWE-89
T2-002	Reflected XSS	XSS	HIGH	7.5	CWE-79
T2-003	Stored XSS in Guestbook	XSS	HIGH	8.8	CWE-79
T2-004	Brute Force	BruteForce	HIGH	8.6	CWE-307
T2-005	CSRF on Password Change	CSRF	HIGH	7.4	CWE-352
T2-006	Local File Inclusion	FileIncl	HIGH	8.2	CWE-98
T2-007	Unrestricted File Upload	FileUpload	CRITICAL	9.8	CWE-434
T2-008	Command Injection in Ping	CmdInj	CRITICAL	9.8	CWE-78
T2-009	Weak DVWA Session Cookie	WeakSession	MEDIUM	6.5	CWE-384
T2-010	DOM-Based XSS	XSS	HIGH	7.5	CWE-79

Table 15: T3: OWASP WebGoat 8.0 (12 vulnerabilities)

ID	Name	Type	Severity	CVSS	CWE
T3-001	Insecure Direct Object Refs	IDOR	HIGH	8.1	CWE-639
T3-002	SQL Injection	SQLi	CRITICAL	9.8	CWE-89
T3-003	XSS (Reflected/Stored/DOM)	XSS	HIGH	7.5	CWE-79
T3-004	Cross-Site Request Forgery	CSRF	HIGH	7.4	CWE-352
T3-005	Server-Side Request Forgery	SSRF	HIGH	8.6	CWE-918
T3-006	Path Traversal (Download)	PathTrav	HIGH	8.1	CWE-22
T3-007	JWT Token Manipulation	JWT	HIGH	8.8	CWE-345
T3-008	Password Reset Flaws	WeakPass	MEDIUM	6.5	CWE-640
T3-009	XML External Entity Injection	XXE	HIGH	9.0	CWE-611
T3-010	Race Conditions	RaceCond	MEDIUM	6.8	CWE-362
T3-011	HTTP Response Splitting	HTTPSplit	MEDIUM	6.5	CWE-93
T3-012	Insecure Deserialization (Java)	Deserial	CRITICAL	9.8	CWE-502

B Evaluation Data Summary

Table 16: Complete evaluation data summary

Run ID	Mode	Dur (min)	V	Conf	Susp	FP	Recall (%)	Cost
ModeA-JuiceShop	A	8.7	0	0	0	0	0.0	$0
ModeA-DVWA	A	8.0	0	0	0	0	0.0	$0
ModeA-WebGoat	A	8.2	0	0	0	0	0.0	$0
ModeB-JuiceShop	B	202.0	12	6	6	0	46.7	$5.00
ModeB-DVWA	B	56.0	8	4	4	0	40.0	$0.57
ModeB-WebGoat	B	200.0	5	5	0	0	41.7	$3.70
ModeC-JuiceShop	C	8.4	0	0	0	0	0.0	$0
ModeC-DVWA	C	8.2	0	0	0	0	0.0	$0
ModeC-WebGoat	C	8.1	0	0	0	0	0.0	$0

C Decision Quality Case Study

A detailed iteration-by-iteration analysis of Mode B's autonomous decision-making on OWASP Juice Shop is available in `DECISION_QUALITY_CASE_STUDY.md`.

Key findings:

- 93 LLM-powered decision cycles analyzed
- Average decision quality score: 6.4/10
- Strongest phase: Exploitation (SQLi chain, IDOR detection)
- Weakest phase: RCE investigation (false positive chasing)
- User constraint ("focus on high bugs only") effectively influenced prioritization

---

Report generated: 2026-03-24
Framework: Baseline Pentesting Evaluation Framework
Phantom Version: v0.9.121
Evaluation data: `experiments/results/evaluation/thesis_evaluation.json`
Plots: `experiments/results/evaluation/plots/fig1-fig8.png`